

Lab 12 Advanced SQL 2

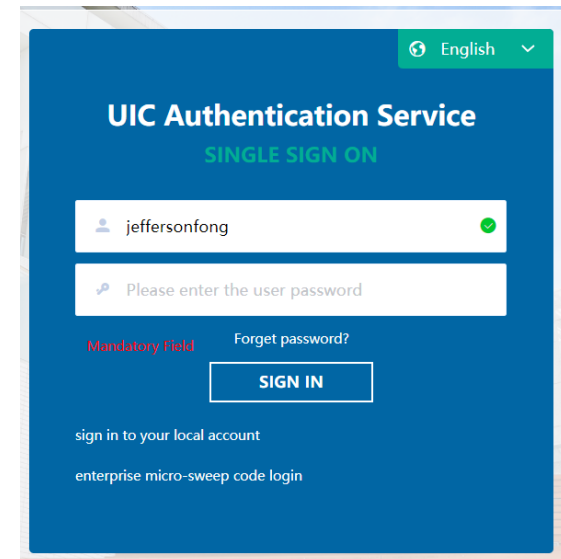
United International College
COMP3013 DBMS

Outline

- **Triggers**
 - Triggers - Event, Condition and Action
 - Triggers in SQL
 - Triggers Example, Row-Level and Statement-Level
 - Tricky Design Issues
 - Triggers Example, More
 - Triggers in MySQL, More
 - Triggers Example with UIC Database
- **Revisions**

Triggers – Event, Condition and Action

- For some simple condition checking within one table, we can use **CHECK**. But, for a complicated predicate crossing multiple tables, we need to use **TRIGGER**.
- A **trigger** is a statement that the system executes automatically as a side effect of a modification to the database.
- Reference: [MySQL reference manual](#) and [5.3] of textbook.
- We need two things:
 - Specify when a trigger is to be executed, event and condition.
 - Specify the actions to be taken when the trigger executes.
- Event-Condition-Action (non-database) example
 - Event – Click “SIGN IN” button
 - Condition1 – User name and password are correct
 - Action1 – Display the user’s UIC webpage
 - A different condition would trigger another action.



The screenshot shows a web interface for the 'UIC Authentication Service'. At the top right, there is a language selector set to 'English'. The main heading is 'UIC Authentication Service' with a sub-heading 'SINGLE SIGN ON'. Below this, there is a username input field containing 'jeffersonfong' with a green checkmark icon to its right. Below the username field is a password input field with a placeholder text 'Please enter the user password'. To the left of the password field, there is a red label 'Mandatory Field'. To the right of the password field, there is a link 'Forget password?'. Below the password field is a blue 'SIGN IN' button. At the bottom of the page, there are two links: 'sign in to your local account' and 'enterprise micro-sweep code login'.

Triggers – Event, Condition and Action

Event-Condition-Action rules:

- When event occurs, check the condition; if true, do the actions.
- Move monitoring logic from apps into DBMS.
- Enforce constraints
 - beyond what constraint system supports and
 - automatic constraint “repair”.

Syntax

- An trigger in SQL takes the form

```
CREATE TRIGGER <trigger-name>  
BEFORE/AFTER <event>  
FOR EACH ROW  
<Trigger body>
```

- **BEFORE** or **AFTER** specifies when the trigger is activated.
- <event> can be **INSERT**, **DELETE**, or **UPDATE**.
- **FOR EACH ROW** is a fixed phrase.
- <Trigger body> specifies the actions will be taken if some conditions are satisfied.

Syntax

The body of a trigger can contains

- **referencing variables**
 - **OLD**.<column name> refers to old rows for deletion or update events.
 - **NEW**.<column name> refers to new rows for insertion or update events.
- **data modification statements**
 - **INSERT**, **DELETE**, or **UPDATE** like other SQL statements
- **Construct**
 - Quoted by **BEGIN** and **END**
 - Details will be given later.

Example with UIC Database

- Referential Integrity

```
ALTER TABLE student ADD CONSTRAINT student_program  
FOREIGN KEY p_code REFERENCES program(p_code)  
ON DELETE CASCADE
```

- which can be implemented using a trigger

```
CREATE TRIGGER delete_program_student  
AFTER DELETE ON program  
FOR EACH ROW  
DELETE FROM student  
      WHERE student.p_code = old.p_code
```

Construct

- Some complicated constraints on multiple tables have to use **BEGIN...END** construct.
- Each statement in the construct must be ended with “;”
- Which creates a conflict to the delimiter of an SQL query.
- Thus, **DELIMITER** is used to change the delimiter before and after a trigger in SQL.
- For example, we change the delimiter from “;” to “|”.

```
DELIMITER |  
<trigger> |  
DELIMITER ;
```


Conditional Statement

- Suppose we want to make sure that every CST student has a GPA at least 1.0
- A conditional statement can be used.
- The entire trigger is

```
DELIMITER |
CREATE TRIGGER cst_student
AFTER INSERT ON student
FOR EACH ROW
BEGIN
    IF new.gpa<1.0 AND new.p_code IN (
        SELECT p_code FROM program WHERE p_name = 'Computer Science'
    ) THEN
        DELETE FROM student WHERE student.id = new.id;
    END IF;
END;|
DELIMITER ;
```

Exercises

- Create trigger(s) to make sure each person has to be either an instructor or a student but not both. (Total and disjoint generalization in the ER diagram.)
- Also create trigger(s) to make sure each person in *borrow* and *contact* already exists in either *instructor* or *student* but not both.
- A student gains the credits from a course if he/she receives passing grade (not 'F') from the course. A student graduates from the college if he/she obtains 130 credits. Create trigger(s) to on *enroll* to detect the graduates and remove those graduates from the database.

Save your queries in a txt file. Rename it as “COMP3013 Lab12 ####.txt”, where “####” is your student ID. And submit it on iSpace. The DDL is 24 hours after the lab.

End of Lab 11